

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Differentiable logic gate networks learn Boolean functions on a fixed predecessor graph, making connectivity a constraint on accuracy at a given gate budget. Learned routing relaxes this constraint but increases training memory. We present LogicConnect, a method that designs structured connectivity before gate-function training. Semantic input pairing preserves image-source relationships, while regular multiscale matchings organize deeper connections under a degree-first selection rule. The construction supports dense and convolutional networks without trainable routing parameters or additional inference operators. Across MNIST, Fashion-MNIST, and CIFAR-10, we evaluate hardened accuracy, training resources, and structural ablations. LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three sizes and three paired seeds. Convolutional S/M models gain 3.26/2.08 points in single-seed tests, although replicated S validation remains inconclusive. Compared with learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory and 80% fewer trainable parameters, at an accuracy cost of 1.84–5.30 points. Degree-preserving ablations identify first-layer structure as the strongest measured dense contributor. These results establish offline connectivity design as a useful accuracy/resource operating point for logic gate networks.

I. INTRODUCTION

Differentiable logic gate networks (DLGNs) train Boolean circuits by optimizing continuous gate relaxations and discretizing the learned functions for inference [1]. Their restricted operators make the connection graph particularly consequential: a two-input gate can combine only the two signals assigned to it. Gate training cannot introduce an absent predecessor, making connectivity a design constraint at a given depth and gate budget.

Existing approaches make this decision either before or during training. Fixed random connections retain a simple graph representation, while learned routing optimizes the predecessors together with the gate functions [2], [3]. Learned routing can improve accuracy, but processing candidate inputs increases the training state even when the final circuit retains only two connections per gate. In our local comparison under two LILogic architectures, Top-32 routing requires 16.0–17.1 times the peak GPU memory of fixed connectivity. This cost motivates a complementary question: how much accuracy can a deliberately designed fixed graph recover before routing itself must be learned?

The relevant structural property extends beyond balanced predecessor use. The native dense random baseline already balances fan-out, yet graphs with the same predecessor degrees need not expose the same combinations of image features. Our dense ablation makes this distinction concrete: randomizing first-layer connections while preserving per-slot predecessor degrees reduces mean validation accuracy by 4.49 percentage points with structured deeper layers. Thus, the identity of the paired signals can matter even when local degree statistics remain unchanged.

We introduce LogicConnect to exploit this design opportunity through semantic input pairing and regular multiscale matchings. Semantic pairs preserve the relationship between encoded bits and their underlying image sources. Deeper matchings organize signal combinations across index scales while controlling predecessor reuse. An ancestry-overlap criterion chooses among equally balanced stages without learning routing parameters. Applying these principles to convolutional DLGNs requires a channel-level construction that preserves shared logic trees and spatial sampling, rather than treating every spatial gate application as an independent dense gate.

Our contributions are threefold:

- We introduce a unified fixed-connectivity construction for dense gates, convolutional channel groups, and their classifiers. The construction runs before training and preserves the gate budget, gate parameterization, and inference operators.
- We show that LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three sizes and three paired seeds. We characterize its accuracy/resource trade-off against learned routing and its application to convolutional models.
- We isolate structural effects with degree-preserving randomizations and simpler stage-selection controls. These experiments identify the dense first-layer contribution and bound the claims about deeper structure, ancestry selection, and transfer.

Convolutional S/M models improve by 3.26/2.08 points in existing single-seed tests, while the three-seed S validation interval includes zero. Learned routing remains more accurate in the matched comparison, establishing where additional training memory still buys accuracy.

II. RELATED WORK

Fixed graphs and architectural structure. Deep DLGNs separate gate-function learning from the fixed predecessor graph, enabling Boolean inference after hardening [1].

Convolutional DLGNs introduce shared logic trees, pooling, and residual initialization to exploit image structure [4]. LogicConnect assigns predecessors within these architectural constraints, preserving the dense or convolutional operators. Degree-preserving controls distinguish the assigned pairs from their degree distribution.

Optimizing the predecessor graph. Connection learning addresses the same graph constraint through a different training problem. Mommen et al. optimize distributions over subsets of candidate predecessors [2]. LILogic Net develops sparse Top-K routing and basis projection to improve the accuracy and implementation efficiency of compact logic networks [3]. These methods can discover connections that a fixed constructor misses, but must process candidate signals during training. The relevant comparison includes accuracy, peak GPU allocation, trainable parameters, and observed training time.

Improving gate optimization. Connectivity is only one source of training difficulty. Mind the Gap uses Gumbel noise and a straight-through estimator to reduce discretization loss [5]. Light DLGNs change the gate parameterization, while WARP uses a Walsh–Hadamard representation of Boolean functions [6], [7]. Multilinear polynomial fitting further examines the redundant optimization directions in a soft mixture of gates [8]. These approaches change how functions are learned on available inputs. LogicConnect changes which inputs are available, leaving the baseline gate-training rule fixed for attribution. Compatibility with another parameterization remains an empirical question, which we examine through matched adapted-WARP controls.

Broader design and deployment choices. BitLogic treats encoding, connectivity, fan-in, and training as interacting design axes for lookup-table-native networks [9]. Output-layer studies likewise show that temperature and aggregation choices affect scalability to more classes [10]. After training, unit tying reduces circuit size through functional approximation and refinement [11]. These studies separate predecessor design from gate representation, output aggregation, and circuit simplification. Their individual gains cannot be added without evaluating the combined system. We therefore separate matched comparisons from published references and adaptations.

III. LOGICCONNECT METHODOLOGY

A. Designing the graph at a fixed budget

LogicConnect constructs the predecessor graph before optimizing gate functions. The objective is to organize signal access while retaining the baseline architecture and training state. For layer l with m_l two-input gates and n_l predecessors, let $I_l \in \{0, \dots, n_l - 1\}^{2 \times m_l}$ contain the connection indices. Gate j receives distinct inputs $a = I_l[0, j]$ and $b = I_l[1, j]$. Its relaxed output follows the baseline parameterization [1]:

$$h_{l,j} = \sum_{q=0}^{15} \text{softmax}(\theta_{l,j})_q \tilde{f}_q(h_{l-1,a}, h_{l-1,b}). \quad (1)$$

Here, $\tilde{f}_q$ is a continuous relaxation of Boolean function q . Training updates θ while holding I_l fixed; hardening selects the

highest-weight function. LogicConnect changes the admissible signal combinations through I_l , without modifying the loss, gate relaxation, or output aggregation.

The constructor represents available signals through graph metadata. We associate predecessor i with structural ancestry A_i and propagate $A_j = A_a \cup A_b$. In dense image inputs, an ancestry label identifies a raw channel-pixel value, so thermometer thresholds of the same value share a label. Ancestry records potential graph access, rather than functional dependence after gates have been learned. Figure 1 separates this construction metadata from the fixed indices retained during training and inference.

B. Semantic pairing at the input

Image encodings expose a distinction that index-only pairing misses: two bits can represent either different image sources or different thresholds of one source. LogicConnect therefore uses channel, position, and threshold metadata to form semantic candidate stages, excluding pairs drawn from the same raw source.

Spatial stages pair positions along horizontal and vertical axes at power-of-two strides up to half the axis length, with cyclic boundaries. At the finest scale, additional stages pair aligned channels and mix thresholds with a spatial displacement. Cycling across stages distributes pairs across these axes before repeatedly using one kind of relation. Within a stage, selection favors the smallest endpoint fan-out sum, then maximum endpoint fan-out, then endpoint indices. The initial search examines up to 64 unused candidates in a seeded affine order; after distinct pairs are exhausted, repeated pairs follow the same degree priority. This bounded selection balances semantic variety against construction cost without claiming a global optimum.

C. Multiscale structure in deeper layers

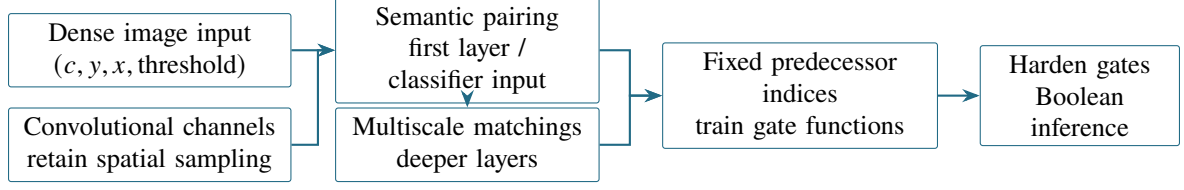
Hidden features no longer have the raw image coordinates needed by the input rule. LogicConnect instead constructs regular matchings over predecessor indices and uses ancestry to distinguish their structural support. A matching changes pair identities while using each predecessor at most once.

For n predecessors, the candidate schedule has $K = \lceil \log_2 n \rceil$ scales. At power-of-two widths, scale s pairs i with $i \oplus 2^s$. Other widths use an affine traversal with a coprime step and pair adjacent elements; odd widths rotate the unmatched predecessor. A complete stage contains $\lfloor n/2 \rfloor$ pairs. A final partial stage selects disjoint pairs by endpoint degree sum, maximum degree, and index, retaining the regular matching structure.

Stage selection gives priority to predecessor balance. Let d_i denote current fan-out and P_s the pairs retained from candidate stage s . The prospective degree spread is

$$D_s = \max_i (d_i + \delta_i(P_s)) - \min_i (d_i + \delta_i(P_s)), \quad (2)$$

where $\delta_i(P_s)$ counts occurrences of i . Minimizing D_s prevents ancestry preferences from overriding balance. Among



Stage priority: degree spread $\rightarrow$ ancestry novelty $\rightarrow$ nominal scale

Fig. 1: One offline construction, two architecture adapters. Semantic pairing defines the input graph; matching stages organize deeper predecessors. Convolution retains the existing spatial sampler and shared logic trees. Construction metadata is discarded before deployment.

Algorithm 1 Deeper-layer stage selection

Require: Ancestry A , width m , layer l , seed
Initialize degrees $d = 0$, output pairs $I = \emptyset$.
while $|I| < m$ **do**
 Generate the K seeded scale matchings M_s .
 Form degree-ordered partial stages P_s if needed.
 Compute D_s and N_s using (2)–(3).
 Among unused scales, minimize D_s , then maximize N_s ;
 break ties by nominal scale order.
 Append P_s ; update d and mark scale s used.
 Reset used scales after a complete K -stage cycle.
end while
Propagate ancestry by union; return fixed indices I .

equally balanced candidates, the constructor favors smaller relative ancestry overlap:

$$N_s = \frac{1}{|Q_s|} \sum_{(a,b) \in Q_s} \left(1 - \frac{|A_a \cap A_b|}{\max(1, \min(|A_a|, |A_b|))} \right). \quad (3)$$

Here, Q_s is the complete candidate matching, or 2,048 evenly indexed pairs for a larger matching, including when only a partial stage will be retained. The score is zero for identical nonempty ancestry and one for disjoint ancestry. LogicConnect minimizes D_s , then maximizes N_s , using cyclic nominal scale order to resolve ties. Every scale is used once per cycle. This criterion is a structural heuristic; its incremental accuracy value is tested separately from that of the complete construction.

For even widths, a complete matching increments every predecessor degree once. A final partial matching increments each degree at most once, giving a final spread of at most one from an initially uniform vector. For example, eight predecessors and ten gates use two complete matchings and two pairs from a third, producing degrees of two or three. Degree-preserving randomization tests the value of pair identities beyond this balance property.

D. Preserving convolutional semantics

Convolutional DLGNs share gate functions across spatial positions, so unrestricted dense rewiring would change the architecture being evaluated. LogicConnect instead assigns channel pairs while retaining the existing spatial receptive-field sampler,

depth-three logic trees, weight sharing, and OR pooling [4]. The first channel assignment uses semantic metadata; subsequent channel groups use the multiscale rule.

Channel ancestry begins with encoded-channel identities. Each output channel receives an additional identity label, preserving distinctions after upstream channel support saturates. Flattened classifier features retain upstream ancestry and receive a spatial-feature identity; semantic pairing then precedes deeper classifier matchings. These labels are construction metadata, not exact pixel-dependence sets. The construction operates at each architecture’s signal granularity.

E. Construction and retained cost

Construction uses packed ancestry bitsets on the CPU. Each stage decision considers K candidate matchings, with overlap work bounded by 2,048 sampled pairs per candidate; ancestry storage still grows with the label universe and layer width. This cost is paid before training and measured separately from GPU allocation. A dense layer retains $16m_l$ gate logits and $2m_l$ fixed indices, with no trainable routing parameters. Deployment retains the hardened functions and connections, while discarding ancestry metadata. Equal logical budgets leave physical area and wire cost dependent on synthesis and placement.

IV. EXPERIMENTAL EVALUATION

A. Evaluation protocol

We evaluate whether fixed graph design improves accuracy (RQ1), changes the training-resource trade-off (RQ2), and yields identifiable structural effects (RQ3). Compatibility and transfer tests examine the boundary of those findings (RQ4). Table I specifies the main MNIST [12], Fashion-MNIST [13], and CIFAR-10 [14] configurations. Each attribution pair matches architecture, input encoding, gate parameterization, optimizer, updates, evaluation cadence, initialization, and training seed. A 10% validation split uses split seed 2027. Checkpoints are selected by hardened validation accuracy and frozen before held-out evaluation.

Experiments use RTX PRO 6000 Blackwell Max-Q GPUs with PyTorch 2.9.0 and CUDA 13.0. We report hardened accuracy, peak PyTorch GPU allocation, trainable parameters, CPU construction time, and training wall time including evaluation. Paired experiments use seeds 0, 1, and 2 unless stated otherwise. Tables report means and sample standard deviations; effect intervals use paired Student- t statistics over

TABLE I: Main evaluation configurations. b : thresholds per raw value; K denotes 1,000. Dense widths enumerate gates per layer; convolutional widths enumerate output channels with $k = 32/256$ for S/M.

Dataset/model	Layer widths	Gate functions	b	Updates	Batch
MNIST	$5 \times 1,334 + 1,330$	8K	1	108K	100
Fashion-MNIST	$5 \times 2,666 + 2,670$	16K	3	108K	100
CIFAR-10 S	$4 \times 12K$	48K	3	108K	100
CIFAR-10 M	$4 \times 128K$	512K	3	108K	100
CIFAR-10 L	$5 \times 256K$	1,280K	5	108K	100
Conv. S/M	$(k, 4k, 16k, 32k)$	83,552 / 668,416	3	350K / 200K	128

Dense: Adam, learning rate 0.01, no augmentation. Convolutional: AdamW, learning rate 0.02, decay 0.002, random crops with four-pixel padding and horizontal flips. All use raw gate logits and validation every 2K updates. Convolutional S/M share gate functions across 874,496/6,995,968 spatial gate applications; each classifier uses three dense layers of widths (1280k, 640k, 320k).

TABLE II: Dense held-out accuracy (T, %). Local entries are mean $\pm$ sample SD over seeds 0, 1, 2. LC gains are percentage points against the paired random baseline.

Model	Gates, K	Random (REP)	LC (OUR)	LC-D (OUR)	Δ LC	Paired 95% CI	Random (R)
MNIST	8	91.27 $\pm$ 0.22	91.94 $\pm$ 0.14	91.91 $\pm$ 0.31	+0.66	[0.02, 1.31]	–
Fashion-MNIST	16	85.20 $\pm$ 0.26	85.72 $\pm$ 0.45	85.91 $\pm$ 0.36	+0.52	[−1.02, 2.06]	–
CIFAR-10 S	48	48.91 $\pm$ 0.35	52.10 $\pm$ 0.63	52.39 $\pm$ 0.30	+3.18	[0.77, 5.60]	51.27 $\pm$ 0.26
CIFAR-10 M	512	54.10 $\pm$ 0.17	58.65 $\pm$ 0.17	58.36 $\pm$ 0.25	+4.56	[3.78, 5.33]	57.39 $\pm$ 0.13
CIFAR-10 L	1,280	55.87 $\pm$ 0.31	60.46 $\pm$ 0.35	61.07 $\pm$ 0.46	+4.59	[3.72, 5.47]	60.78 $\pm$ 0.12

LC wins 3/3 pairs at each coordinate. LC-D: separate dense specialization. Bold: largest local mean. R: published random baseline [1]; no exact-budget MNIST/Fashion reference.

TABLE III: Convolutional CIFAR-10 accuracy and measured training cost. Single-seed test results and three-seed validation replications are separate evidence.

Size	Method	Scope / seeds	Accuracy, %	Time, h	GPU, GiB	Random T (R)
S	Random (REP)	T / 1	57.37	4.98	1.831	60.38
S	LC (OUR)	T / 1	60.63	4.95	1.831	–
M	Random (REP)	T / 1	69.57	25.47	14.614	71.01
M	LC (OUR)	T / 1	71.65	35.69	14.614	–
S	Random (REP)	V / 3	59.82 $\pm$ 1.13	4.97	1.831	–
S	LC (OUR)	V / 3	61.05 $\pm$ 1.04	4.97	1.831	–

R: published S/M results [4]. Full-S V: paired gain +1.23 pp, 95% CI [−3.36, 5.82], 2/3 wins; extra seeds have no test results. Time: mean; memory: maximum peak allocation.

training seeds. The labels OUR, REP, and R distinguish our constructions, locally reproduced baselines, and reported literature values; V/T distinguish validation/test accuracy.

LogicConnect (LC) denotes the unified method of Section III. LogicConnect-D (LC-D) is a separate dense specialization using semantic butterfly pairing and ancestry-based swaps. Its measurements remain separate from the unified method. Bold-face marks the highest local mean, not statistical superiority or an evaluated variant-selection policy.

B. Accuracy at a fixed architectural budget

Table II answers RQ1 across five dense configurations. LogicConnect improves CIFAR-10 accuracy by 3.18, 4.56, and 4.59 points at S, M, and L. All three paired confidence intervals exclude zero, and every seed pair improves. Figure 2 shows the advantage across sizes and paired seeds. MNIST improves by

0.66 points with a positive interval; Fashion-MNIST gains 0.52 points, but its interval includes zero.

The dense specialization provides a useful qualification. LC-D has the larger mean at Fashion-MNIST and CIFAR-10 S/L, while unified LC leads at MNIST and CIFAR-10 M. At L, LC-D improves on LC by 0.61 points. Structure improves on native random wiring, but the unified constructor does not dominate every specialization. Published values use different protocols and remain separate.

Table III reports convolutional S/M test gains of 3.26/2.08 points at unchanged architecture, using one seed each. Full-S validation replication gives 61.05% for LC versus 59.82% for random, with a paired gain of 1.23 points and interval [−3.36, 5.82]. LC wins two of three seeds. The held-out comparisons demonstrate positive operating points, while validation

TABLE IV: CIFAR-10 routing trade-off under LILogic architectures: M is $1 \times 64\text{K}$ gates; L is $2 \times 128\text{K}$ gates. Both use seven input thresholds. Local accuracy is T; R retains the published statistic.

Size	Method	n	Accuracy, %	Params., M	Time, min	GPU, GiB	Accuracy (R)
M	Random (REP)	3	49.01 ± 0.43	1.024	14.62	0.474	49.17 ± 0.14
M	LC (OUR)	3	52.54 ± 0.30	1.024	14.59	0.474	–
M	Top-32 (REP)	1	57.84	5.120	37.81	8.100	57.28 ± 0.30
L	Random (REP)	3	55.33 ± 0.47	4.096	15.20	1.557	54.76 ± 0.27
L	LC (OUR)	3	60.19 ± 0.29	4.096	18.47	1.557	–
L	Top-32 (REP)	1	62.03	20.480	350.63	24.861	60.98 ± 0.19

Local: 35K updates, batch 256, Adam at 0.075, reflected-padding crops and horizontal flips. R: LILogic v2 [3]. Top-32 adds 4.096M/16.384M routing parameters; its single-seed resource comparison is descriptive.

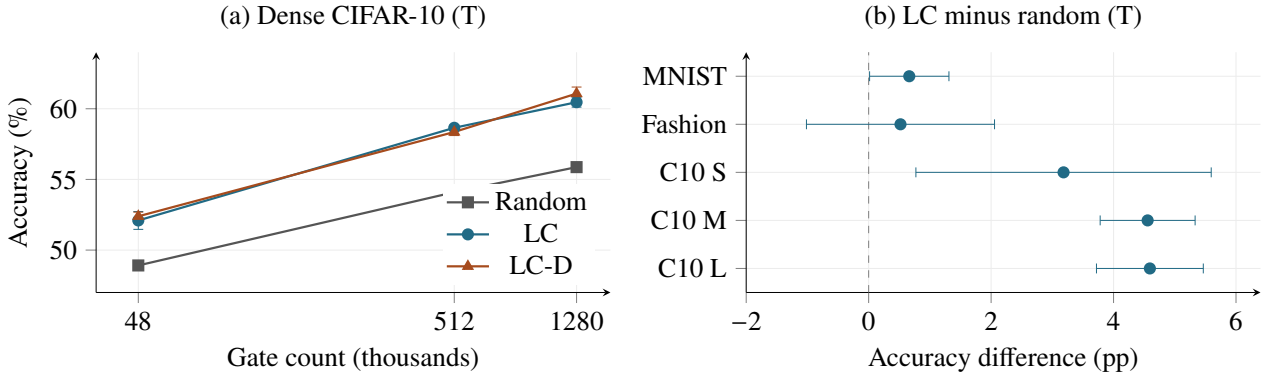


Fig. 2: Dense accuracy at fixed budgets. (a) Mean $\pm$ sample SD over three seeds. LC-D is the separate dense specialization. (b) Paired 95% confidence intervals for LC minus random; all three CIFAR-10 intervals exclude zero.

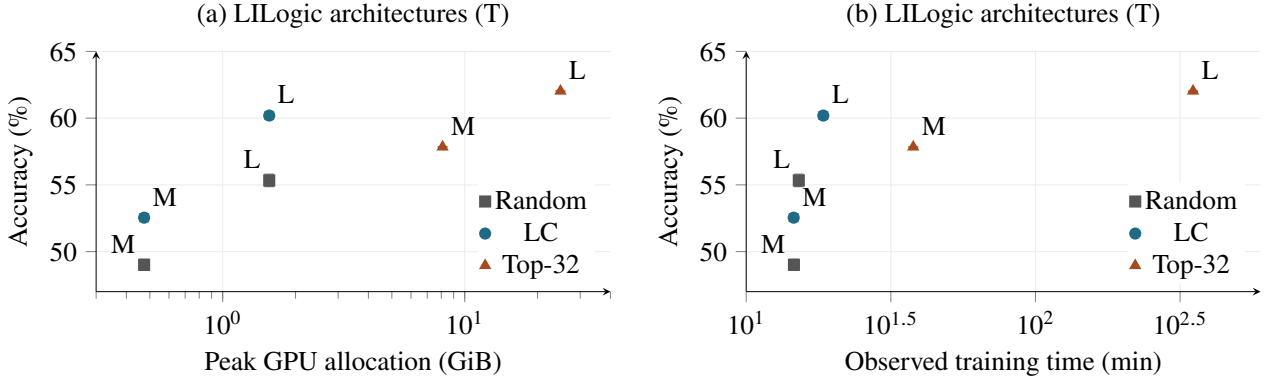


Fig. 3: Accuracy versus (a) peak GPU allocation and (b) observed training time under the LILogic M/L configurations. LC and random: three seeds, mean $\pm$ sample SD. Top-32: one seed. M/L identify architectures; wall-time ratios are descriptive execution measurements.

replication does not establish a consistent gain. The additional checkpoints have no held-out evaluations.

C. Accuracy versus training resources

Table IV addresses RQ2 under the two LILogic architectures and their seven-threshold encoding. At M/L, LC attains 52.54/60.19% test accuracy, improving on matched fixed random by 3.53/4.86 points. Learned Top-32 routing remains

more accurate at 57.84/62.03%, but its peak allocations are 8.10/24.86 GiB versus 0.474/1.557 GiB for LC. The fixed constructor therefore uses 94.2%/93.7% less peak GPU memory and 80% fewer trainable parameters, at an accuracy cost of 5.30/1.84 points.

Figure 3 presents this trade-off in memory and time. Observed Top-32-to-LC training-time ratios are 2.59/18.98 at M/L;

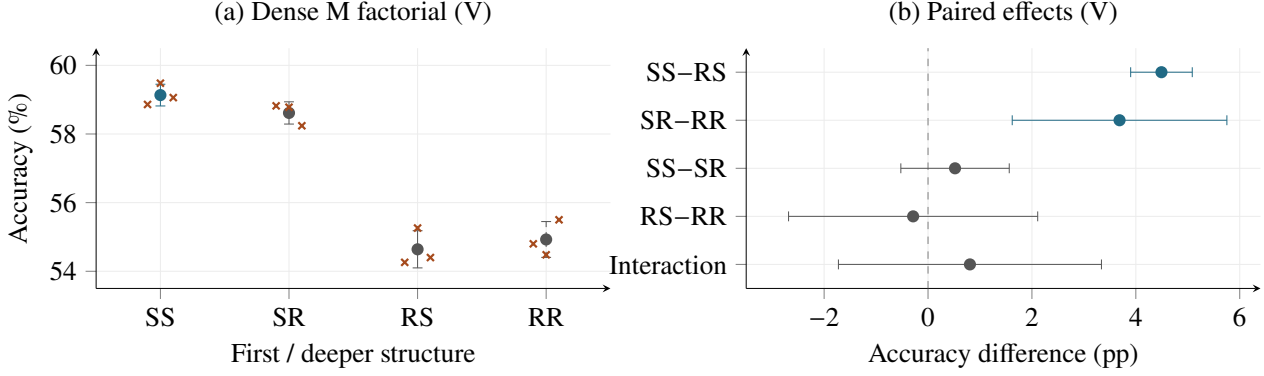


Fig. 4: Dense M attribution at 20K updates. S retains LC structure; R randomizes pairs while preserving each predecessor degree in each input slot. (a) Three seed values (crosses) and mean $\pm$ sample SD. (b) All five paired 95% intervals, exploratory and unadjusted. The first two contrasts isolate first-layer structure.

they describe recorded executions, not isolated algorithmic speedups. At L, LC recovers 4.86 of the 6.70 accuracy points that Top-32 gains over random, approximately 73%, using about one-sixteenth of its peak allocation. This operating point suits constrained training memory, while retaining the accuracy cost explicitly.

At dense M/L, CPU construction takes 14.33/31.22 seconds, compared with 41.39/113.53 minutes of training. These construction times are below 0.6% of the corresponding recorded training durations. Dense matched runs retain the same peak GPU allocation as random. Full-S convolutional training averages approximately 4.97 hours for both methods at 1.83 GiB, whereas full-M records 35.69 hours for LC and 25.47 for random. Different execution conditions prevent attributing the M timing difference solely to connectivity. No physical area, energy, or placement benefit is inferred from equal gate counts.

D. Structural attribution

RQ3 asks whether the gain survives controls that preserve predecessor use. At dense M with 20K updates, we independently keep or randomize first-layer and deeper-layer wiring. Randomization preserves each predecessor’s degree in each input slot. The SS, SR, RS, and RR arms reach 59.13, 58.61, 54.64, and 54.93% validation accuracy, respectively, where S denotes LC structure.

Figure 4 separates the paired effects. First-layer structure adds 4.49 points with structured deeper layers and 3.69 with randomized deeper layers; both intervals exclude zero. Adding deeper structure to the structured first layer gives 0.52 points with interval $[-0.52, 1.56]$. The interaction interval also includes zero. The full construction leads in mean accuracy; first-layer pairing has the strongest identified effect.

Simpler stage-selection controls bound the explanation further. At 20K updates, LC-minus-nominal gains are -0.09 points for dense M and $+0.59$ for convolutional S, with intervals $[-1.76, 1.59]$ and $[-1.12, 2.31]$. Neither result establishes an incremental ancestry-selection advantage. A convolutional body/classifier ablation finds a 2.14-point body effect with a

fixed structured classifier, interval $[0.67, 3.61]$. That classifier retains reference-body ancestry, so the result is conditional rather than a head-independent body effect. All ablation intervals are exploratory and unadjusted.

E. Compatibility and transfer boundaries

RQ4 tests whether benefits survive changes outside the main graph comparison. Matched adapted-WARP experiments retain a dense LC gain of 4.70 points, interval $[3.92, 5.48]$, but the convolutional gain is 0.09 with interval $[-1.24, 1.42]$. Both adapted recipes trail their matched raw parameterizations. Parameter reduction alone does not establish a better accuracy/memory result.

Frozen CIFAR-10.1 evaluation [15] provides a separate distribution test without adaptation. Dense M gains 3.73 points, interval $[2.80, 4.67]$, across three seeds. Convolutional S/M changes are $-0.05/+1.95$ points, one seed each, while CIFAR-100 pilots do not establish an LC benefit. The dense transfer gain extends beyond the original test set; the other results bound its applicability.

V. CONCLUSION

LogicConnect treats fixed connectivity as an architectural design choice before gate-function training. Semantic input pairs organize access to image sources, while regular multiscale matchings extend the construction to deeper predecessors and convolutional channel groups. The graph retains the baseline gate budget and inference operators.

This design improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across the three evaluated sizes. Against learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory at an accuracy cost of 1.84–5.30 points. This establishes an accuracy/resource choice for DLGN training.

Degree-preserving ablations locate the strongest dense effect in the first layer. Deeper structure, ancestry selection, and replicated convolutional gains remain unresolved. Broader claims require further held-out replications and physical implementation measurements.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, “Deep differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [2] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, “A method for optimizing connections in differentiable logic gate networks,” arXiv:2507.06173, 2025.
- [3] K. Fojcik, R. Zioma, and J. Armaitis, “LILogic Net: Compact logic gate networks with learnable connectivity for efficient hardware deployment,” arXiv:2511.12340v2, 2026.
- [4] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, “Convolutional differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [5] S. Yousefi, A. Plesner, T. Aczel, and R. Wattenhofer, “Mind the gap: Removing the discretization gap in differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, 2025.
- [6] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, “Light differentiable logic gate networks,” arXiv:2510.03250, 2025.
- [7] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, “WARP logic neural networks,” arXiv:2602.03527, 2026.
- [8] Y. Kim, “Fitting multilinear polynomials for logic gate networks,” arXiv:2605.08657, 2026.
- [9] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, “BitLogic: A framework for gradient-based LUT-native neural networks,” *Transactions on Machine Learning Research*, 2026.
- [10] S. Brändle, T. Aczel, A. Plesner, and R. Wattenhofer, “From MNIST to ImageNet: Understanding the scalability boundaries of differentiable logic gate networks,” arXiv:2509.25933, 2025.
- [11] S. Lee, J. Sun, and J. Chung, “Two-stage unit tying for simplifying differentiable logic gate networks,” in *Proceedings of the International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026.
- [12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [13] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms,” arXiv:1708.07747, 2017.
- [14] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [15] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do CIFAR-10 classifiers generalize to CIFAR-10?” arXiv:1806.00451, 2018.